Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Qt İleri Bileşenler

Bölüm 7 – Hafta 7

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 7. Hafta

Bu Haftanın Planı

1. Ünite 1: Qt Kaynak Sistemi
2. Ünite 2: Qt Stil Sistemi
3. Ünite 3: Ayarlar ve Veri Kalıcılığı
4. Ünite 4: Olay İşleme ve Zamanlayıcılar
5. Ünite 5: QDataStream ile Binary Veri Kaydetme
6. Özet

Ünite 1: Qt Kaynak Sistemi

Qt Kaynak Sistemi – Neden?

Sorun

- İkonlar ayrı dosya olarak dağıtılır
- Kullanıcı yanlışlıkla silebilir
- Yol değişirse uygulama çöker

Çözüm: .qrc

- Kaynaklar binary'ye gömülür
- Kaybolma riski yok
- :/ ön eki ile erişim

.qrc Dosya Yapısı

```
<RCC>
    <qresource prefix="/icons">
        <file>ekle.png</file>
        <file>sil.png</file>
        <file>kaydet.png</file>
    </qresource>
    <qresource prefix="/stiller">
        <file>karanlik.qss</file>
    </qresource>
</RCC>
```

CMake Entegrasyonu

`set(CMAKE_AUTORCC ON)` + `qt_add_executable(app main.cpp resources.qrc)`

QIcon ve QPixmap

QIcon – Widget İkonları

```
QIcon ikon(":/icons/ekle.png");
buton->setIcon(ikon);
buton->setIconSize(QSize(24,24));
```

QPixmap – Görseller

```
QPixmap logo(":/images/logo.png");
label->setPixmap(
  logo.scaled(128, 128,
    Qt::KeepAspectRatio));
```

İkonlu Action

```
auto* act = new QAction(QIcon(":/icons/kaydet.png"),
                        "Kaydet", this);
```

Ünite 2: Qt Stil Sistemi

QSS – Qt Style Sheets

CSS benzeri söz dizimi ile widget görünümü özelleştirme:

```
QPushButton {
    background-color: #008080;
    color: white;
    border-radius: 4px;
    padding: 6px 12px;
}

QPushButton:hover {
    background-color: #009090;
}
```

QSS Seçicileri

Seçici Türleri

`QPushButton`	Tüm butonlar (tür)
`#ekleButon`	Nesne adı "ekleButon"
`QDialog QPushButton`	QDialog içindeki butonlar
`QPushButton:hover`	Fare üzerindeyken
`QPushButton:pressed`	Basılı durumda

Öncelik Kuralı

Daha spesifik seçiciler genel olanları ezer (CSS cascade).

Tema Değiştirme

```
void temaUygula(const QString& qssYol) {
    QFile dosya(qssYol);
    if (dosya.open(QFile::ReadOnly | QFile::Text)) {
        qApp->setStyleSheet(dosya.readAll());
    }
}

// Kaynak sisteminden yükleme
temaUygula(":/stiller/karanlik.qss");
```

TemaYöneticisi

- Merkezi tema yönetim sınıfı
- `temaDeğişti()` sinyali
- `QActionGroup` ile radyo menü

Ünite 3: Ayarlar ve Veri Kalıcılığı

QSettings – Uygulama Ayarları

```
QSettings ayarlar;
ayarlar.setValue("tema", "karanlık");
ayarlar.setValue("pencere/genislik", 1024);

QString tema = ayarlar.value("tema", "aydınlık")
                      .toString();
```

Platform Depolama

Windows	Registry
macOS	.plist dosyaları
Linux	.conf dosyaları

Pencere Durumu Kaydetme

```
// Kapanışta kaydet
void closeEvent(QCloseEvent* event) {
    QSettings ayarlar;
    ayarlar.setValue("pencere/geometri",
                     saveGeometry());
    ayarlar.setValue("pencere/durum",
                     saveState());
}

// Açılışta yükle
restoreGeometry(
    ayarlar.value("pencere/geometri").toByteArray());
```

QJsonDocument – JSON İşlemleri

```
// Yazma
QJsonObject ogr;
ogr["ad"] = "Ali Yılmaz";
ogr["gpa"] = 3.45;

QJsonArray dizi;
dizi.append(ogr);

QJsonDocument doc(QJsonObject{{"öğrenciler", dizi}});
dosya.write(doc.toJson(QJsonDocument::Indented));
```

```
// Okuma
QJsonDocument doc = QJsonDocument::fromJson(data);
QJsonArray dizi = doc.object()["öğrenciler"].toArray();
```

Son Açılan Dosyalar

```
void sonDosyaEkle(const QString& yol) {
    QSettings ayarlar;
    QStringList dosyalar = ayarlar.value(
        "sonDosyalar").toStringList();
    dosyalar.removeAll(yol);
    dosyalar.prepend(yol);
    while (dosyalar.size() > 5)
        dosyalar.removeLast();
    ayarlar.setValue("sonDosyalar", dosyalar);
}
```

Ünite 4: Olay İşleme ve Zamanlayıcılar

Qt Olay Sistemi

Olay Döngüsü

1. İşletim sistemi olay üretir
2. Qt → `QEvent` nesnesi
3. Olay → widget'ın `event()` metoduna
4. Widget olayı işler veya üst sınıfa aktarır

Sık Kullanılan Olaylar

`mousePressEvent()`	Fare tıklaması
`keyPressEvent()`	Tuş basma
`closeEvent()`	Pencere kapatma
`resizeEvent()`	Boyut değişimi

QTimer – Zamanlayıcılar

Periyodik Timer

```
auto* timer = new QTimer(this);
connect(timer, &QTimer::timeout,
  this, &MW::otomatikKaydet);
timer->start(30000); // 30 sn
```

Tek Seferlik

```
QTimer::singleShot(3000, this,
  [this]() {
    statusBar()->clearMessage();
  });
```

Dikkat

`QTimer` callback'i UI thread'inde çalışır. Ağır işlemler arayüzü dondurur!

QUndoStack – Geri Al/Yinele

```
class EkleKomutu : public QUndoCommand {
public:
    void redo() override { model->ogrenciEkle(ogr); }
    void undo() override { model->sonOgrenciyiSil(); }
};

// Komut ekle (otomatik redo çağrılır)
m_undoStack->push(new EkleKomutu(model, ogr));

// Menü action'ları
auto* geriAl = m_undoStack->createUndoAction(this);
geriAl->setShortcut(QKeySequence::Undo);
```

Değişiklik Takibi

`setClean()` ile kayıt noktası, `cleanChanged()` sinyali ile "*" gösterimi.

Ünite 5: QDataStream ile Binary Veri Kaydetme

JSON ve Binary Karşılaştırması

JSON (Ünite 3)

- Metin tabanlı, okunabilir
- Parse gerektirir → yavaş
- Büyük dosya boyutu
- Web uyumlu

Binary (QDataStream)

- Kompakt format
- Doğrudan okuma → hızlı
- Küçük dosya boyutu
- Tip güvenliği güçlü

Ne Zaman Hangisi?

Dış sistemlerle paylaşım → JSON | İç depolama / performans → Binary

QDataStream – Temel Kullanım

Yazma

```
QFile dosya("veri.dat");
dosya.open(QIODevice::WriteOnly);
QDataStream out(&dosya);
out.setVersion(QDataStream::Qt_6_0);

int sayi = 42;
QString isim = "Ali";
out << sayi << isim;
```

Okuma

```
QFile dosya("veri.dat");
dosya.open(QIODevice::ReadOnly);
QDataStream in(&dosya);
in.setVersion(QDataStream::Qt_6_0);

int sayi;
QString isim;
in >> sayi >> isim;
```

Kritik Kural

Veriler **yazıldığı sırayla** okunmalıdır. Binary formatta alan adı yoktur!

Özel Sınıf Serileştirme

```
struct Ogrenci {
    QString numara, ad, bolum;
    double gpa = 0.0;

    friend QDataStream& operator<<(
            QDataStream& out, const Ogrenci& ogr) {
        out << ogr.numara << ogr.ad
            << ogr.bolum << ogr.gpa;
        return out;
    }

    friend QDataStream& operator>>(
            QDataStream& in, Ogrenci& ogr) {
        in >> ogr.numara >> ogr.ad
           >> ogr.bolum >> ogr.gpa;
        return in;
    }
};
```

Kullanım

OBYS Binary Dosya Formatı (.obys)

Dosya Yapısı

Magic Number (0x4F425953) → Versiyon → Kayıt Sayısı → Öğrenci Verileri

```
constexpr quint32 OBYS_MAGIC   = 0x4F425953; // "OBYS"
constexpr quint16 OBYS_VERSION = 1;

bool binaryKaydet(const QString& yol,
                  const QList<Ogrenci>& liste) {
    QFile dosya(yol);
    if (!dosya.open(QIODevice::WriteOnly)) return false;
    QDataStream out(&dosya);
    out.setVersion(QDataStream::Qt_6_0);
    out << OBYS_MAGIC << OBYS_VERSION;
    out << static_cast<quint32>(liste.size());
    for (const auto& ogr : liste) out << ogr;
    return true;
}
```

Binary Yükleme – 3 Katmanlı Güvenlik

```
QList<Ogrenci> binaryYukle(const QString& yol) {
    QList<Ogrenci> liste;
    QFile dosya(yol);
    if (!dosya.open(QIODevice::ReadOnly)) return liste;
    QDataStream in(&dosya);
    in.setVersion(QDataStream::Qt_6_0);

    quint32 magic; in >> magic;
    if (magic != OBYS_MAGIC) return liste;        // 1) Format

    quint16 ver; in >> ver;
    if (ver > OBYS_VERSION) return liste;          // 2) Versiyon

    quint32 sayi; in >> sayi;
    for (quint32 i = 0; i < sayi; ++i) {
        Ogrenci ogr; in >> ogr;
        if (in.status() != QDataStream::Ok) break; // 3) Veri
        liste.append(ogr);
    }
    return liste;
}
```

Uzantıya Göre Otomatik Format Seçimi

```
void MainWindow::kaydet(const QString& yol) {
    if (yol.endsWith(".json"))
        jsonKaydet(yol, ogrenciler);     // Unite 3
    else if (yol.endsWith(".obys"))
        binaryKaydet(yol, ogrenciler);   // Unite 5
}
```

Performans Farkı (100 Öğrenci)

	JSON	**Binary**
Dosya boyutu	∼15 KB	∼4 KB
Okuma/yazma hızı	1x	3–5x

Özet

Bölüm 7 Özet

Öğrendiklerimiz

1. **Kaynak Sistemi**: .qrc, QIcon, QPixmap, araç çubuğu ikonları
2. **Stil Sistemi**: QSS, seçiciler, karanlık/aydınlık tema
3. **Veri Kalıcılığı**: QSettings, QJsonDocument, son açılanlar
4. **Olay ve Timer**: Event loop, QTimer, QUndoStack
5. **QDataStream**: Binary serileştirme, magic number, .obys formatı

OBYS Durumu

İkonlu arayüz + Tema + JSON/Binary kayıt + Undo/Redo → Profesyonel!

Gelecek Hafta: Bölüm 8 – OBYS GUI Tamamlama

QChart, yazdırma, dışa aktarma ile OBYS'u bitireceğiz.